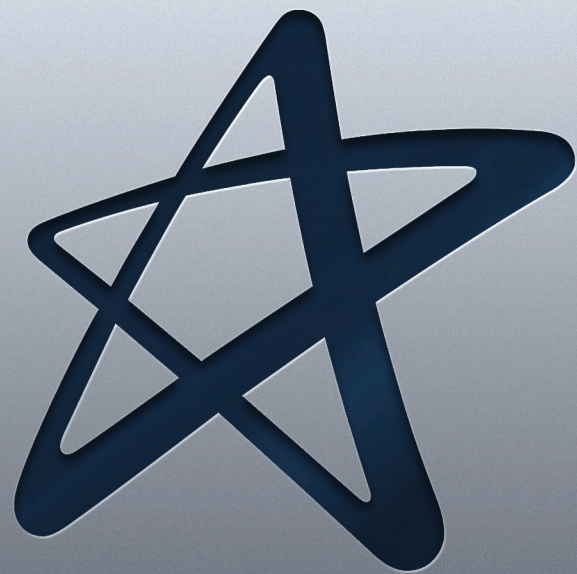


Computação Paralela e Distribuída



Cruzeiro do Sul Virtual
Educação a distância

Material Teórico



Comunicação entre Processos

Responsável pelo Conteúdo:

Prof. Esp. Allan Piter Pressi

Revisão Textual:

Prof.^a Me. Sandra Regina Fonseca

UNIDADE

Comunicação entre Processos



- Interprocessos de Comunicação;
- RPC - Chamada de Procedimento Remoto;
- Comunicação Indireta.



OBJETIVO DE APRENDIZADO

- Compreender a interação entre processos do sistema e dispositivos conectados, compreender que, para que a comunicação ocorra, uma série de elementos e recursos são utilizados, considerando que cada processo deve possuir uma interação com seus outros recursos, bem como prevenir falhas nessa comunicação e também aspectos de segurança a operação.

Orientações de estudo

Para que o conteúdo desta Disciplina seja bem aproveitado e haja maior aplicabilidade na sua formação acadêmica e atuação profissional, siga algumas recomendações básicas:



Assim:

- ✓ Organize seus estudos de maneira que passem a fazer parte da sua rotina. Por exemplo, você poderá determinar um dia e horário fixos como seu “momento do estudo”;
- ✓ Procure se alimentar e se hidratar quando for estudar; lembre-se de que uma alimentação saudável pode proporcionar melhor aproveitamento do estudo;
- ✓ No material de cada Unidade, há leituras indicadas e, entre elas, artigos científicos, livros, vídeos e *sites* para aprofundar os conhecimentos adquiridos ao longo da Unidade. Além disso, você também encontrará sugestões de conteúdo extra no item **Material Complementar**, que ampliarão sua interpretação e auxiliarão no pleno entendimento dos temas abordados;
- ✓ Após o contato com o conteúdo proposto, participe dos debates mediados em fóruns de discussão, pois irão auxiliar a verificar o quanto você absorveu de conhecimento, além de propiciar o contato com seus colegas e tutores, o que se apresenta como rico espaço de troca de ideias e de aprendizagem.

Interprocessos de Comunicação

Esta unidade trata das características dos protocolos de comunicação entre processos em um sistema distribuído - isto é, comunicação entre processos.

A comunicação entre processos na Internet fornece um datagrama e fluxo de comunicação. As APIs são apresentadas, juntamente com uma discussão de modelos de falha. Eles fornecem blocos de construção alternativos para protocolos de comunicação.

Isto é complementado por um estudo de protocolos para a representação de coleções de dados objetos em mensagens e de referências a objetos remotos. Juntos, esses serviços oferecem apoio à construção de serviços de comunicação de alto nível.

Os processos de comunicação são capazes de enviar uma mensagem de um remetente para um grupo de receptores. A unidade também considera a comunicação *multicast*, incluindo *Multicast IP* e os principais conceitos de confiabilidade e ordenação de mensagens em multicast comunicação.

O *multicast* é um requisito importante para aplicativos distribuídos e deve ser fornecido mesmo que o suporte subjacente para multicast IP não esteja disponível. Isso é tipicamente fornecido por uma rede de sobreposição construída sobre a rede TCP/IP.

As redes de sobreposição também podem fornecer suporte para compartilhamento de arquivos, confiabilidade e distribuição de conteúdo. O *Message Passing Interface* (MPI) é um padrão desenvolvido para fornecer uma API para um conjunto de operações de passagem de mensagens com variantes síncronas e assíncronas.

Introdução

Neste item são abordadas as preocupações com os aspectos de comunicação de *middleware*, embora os princípios discutidos sejam mais amplamente aplicáveis. Esta questão é associada com o design dos componentes.

Vamos discutir os chamados remotos das aplicações e as questões em que se preocupa com paradigmas indiretos de comunicação. Serão discutidos os protocolos de nível de transporte da Internet UDP e TCP, bem como o *middleware* e programas aplicativos poderiam usar esses protocolos.

Nas próximas seções deste item, apresentamos as características da comunicação entre processos e, em seguida, discute-se UDP e TCP do ponto de vista de um programador juntamente com uma discussão sobre sua falha.

Modelos de Comunicação

A interface de programas aplicativos para UDP fornece uma mensagem passando uma forma de abstração, a forma mais simples de comunicação entre processos. Isso permite que um processo de envio e transmitir uma única mensagem para um processo de recebimento.

Os pacotes independentes contendo essas mensagens são chamados de datagramas. A interface de um programa aplicativo para TCP pode fornecer a abstração de um fluxo entre pares de processos.

A informação comunicada consiste em um fluxo de itens de dados sem limites de mensagens. Os fluxos fornecem um bloco de construção para comunicação produtor-consumidor.

Um produtor e um consumidor formam um par de processos em que o papel do primeiro é produzir itens de dados e o papel do segundo é consumi-los.

Ainda neste item, vamos tratar sobre como os objetos e as estruturas de dados usados e programas de aplicação podem ser traduzidos em uma forma adequada para enviar mensagens em rede, levando em conta o fato de que computadores diferentes podem usar representações para itens de dados simples. Também se discute uma representação adequada para referências de objeto em um sistema distribuído.

Vamos ainda compreender a comunicação multicast: uma forma de interprocesso de comunicação em que um processo em um grupo de processos transmite a mesma mensagem para todos os membros do grupo. Depois de explicar o *multicast IP*, vamos aprender sobre a necessidade de formas mais confiáveis de *multicast*.

Comunicação Síncrona e Assíncrona

Uma fila está associada a cada destino da mensagem. Os processos de envio fazem com que as mensagens sejam adicionadas a filas remotas e processos de recebimento removem mensagens de filas locais. A comunicação entre os processos de envio e recebimento pode ser síncrona ou assíncrona.

Na forma síncrona de comunicação, os processos de envio e recebimento são sincronizados em toda a mensagem. Nesse caso, enviar e receber são operações de bloqueio. Sempre que um enviar é emitido o processo de envio (ou *thread*) é bloqueado até que o recebimento correspondente é confirmado.

Na forma assíncrona de comunicação, no uso da operação de envio não ocorre o bloqueio, sendo permitido para continuar assim que a mensagem tiver copiado para um buffer local e a transmissão da mensagem continua em paralelo com o processo de envio. A operação de recebimento pode ter bloqueio e não bloqueio variante.

Em um ambiente de sistema como o Java, que suporta vários encadeamentos em um único processo, o bloqueio de recebimento não tem desvantagens, pois pode ser emitido por um thread enquanto outros threads no processo permanecem ativos, sendo a simplicidade de sincronizar ou receber threads com a mensagem recebida uma vantagem substancial.

Sem bloqueio, a comunicação parece ser mais eficiente, mas envolve complexidade extra no processo de recebimento associado à necessidade de adquirir a mensagem de entrada fluxo de controle. Por estas razões, os sistemas atuais geralmente não fornecem forma de bloqueio de receber.

Nos protocolos da Internet, as mensagens são enviadas para pares (endereço Internet, porta local). Uma porta local é um destino de mensagem dentro de um computador, especificado como um número inteiro. Uma porta tem exatamente um receptor, podendo, contudo, em ambas as formas de comunicação (UDP e TCP), usar a abstração de *socket*, que fornece um ponto final para comunicação entre processos.

Sockets são originários de sistemas UNIX, mas também estão presentes na maioria das outras versões do UNIX, incluindo o Linux, o Windows e o SO Macintosh. A comunicação entre processos consiste em transmitir uma mensagem entre um *socket* em um processo e um *socket* em outro processo. Para um processo receber mensagens, seu *socket* deve ser ligado para uma porta local e um dos endereços da Internet do computador em que é executada.

Comunicação do Datagrama UDP

Um datagrama enviado pelo protocolo UDP é transmitido de um processo de envio para um processo de recebimento sem confirmação ou tentativas. Se ocorrer uma falha, a mensagem pode não chegar.

Um datagrama é transmitido entre processos quando um processo o envia e outro recebe. Para enviar ou receber mensagens, um processo deve primeiro criar um *socket* vinculado a um Endereço da Internet do *host* local e de uma porta local.

Um servidor ligará seu *socket* a um servidor e porta, um cliente liga seu *socket* a qualquer porta local livre. O método de recebimento retorna o endereço da Internet e porta do remetente, além da mensagem, permitindo que o destinatário envie uma resposta.

A seguir, alguns problemas relacionados à comunicação UDP:

- Tamanho da mensagem;
- Receber de qualquer origem;
- Omissão das falhas.

Uso de UDP

Para algumas aplicações, é aceitável usar um serviço em que falhas ocasionais de omissão podem acontecer. Por exemplo, o DNS (*Domain Name System*), que procura Nomes DNS na Internet, é implementado em UDP.

Voz sobre IP (VOIP) também é executada sobre UDP. Os datagramas UDP são, às vezes, uma opção atraente porque não sofrem com as despesas gerais associadas à entrega garantida de mensagens.

Há três principais fontes de sobrecarga:

- A necessidade de armazenar informações de estado na origem e no destino;
- A transmissão de mensagens extras;
- Latência para o remetente.

Comunicação de Fluxo TCP

O protocolo TCP, que se origina do BSD 4.x UNIX, fornece a abstração de um fluxo de bytes para o qual os dados podem ser gravados e dos quais os dados podem ser lidos.

As seguintes características da rede estão ocultas pelo fluxo abstração:

- Tamanhos de mensagem;
- Garantia de Entrega;
- Controle de fluxo;
- Destinos da mensagem.

A seguir, estão alguns problemas pendentes relacionados à comunicação de fluxo:

- Correspondência de itens de dados;
- Bloqueio;
- *Threads*.

Modelo de Falha TCP

Para satisfazer a propriedade de integridade de uma comunicação confiável, o TCP fluxos usa somas de verificação para detectar e rejeitar pacotes corrompidos, e números de sequência para detectar e rejeitar pacotes duplicados. Por uma questão de propriedade de validade, os fluxos TCP usam *timeouts* e retransmissões para lidar com pacotes perdidos. Portanto, as mensagens são garantidas para serem entregues mesmo quando alguns dos pacotes subjacentes são perdidos. Mas se a perda de pacotes em uma conexão ultrapassar algum limite, ou se a rede que

conecta um par de processos de comunicação for interrompida, ou se torna severamente congestionada, o *software* TCP responsável pelo envio de mensagens não receberá confirmações e depois de um tempo irá declarar a conexão a ser quebrada. Assim TCP não fornece comunicação confiável, porque não garante a entrega de mensagens diante de todas as dificuldades possíveis.

Quando uma conexão é interrompida, um processo que a utiliza será notificado se tentar ler ou escrever. Isso tem os seguintes efeitos:

- Os processos que usam a conexão não podem distinguir entre falha de rede e falha do processo na outra extremidade da conexão.
- Os processos de comunicação não podem dizer se as mensagens que enviaram recentemente foram recebidas ou não.

Usos do TCP

Muitos serviços frequentemente usados são executados em conexões TCP, com números de porta. Estes incluem os seguintes:

- **HTTP:** o protocolo de transferência de hipertexto é usado para comunicação entre navegadores e servidores web.
- **FTP:** O Protocolo de Transferência de Arquivos permite que diretórios em um computador remoto sejam navegáveis e arquivos sejam transferidos de um computador para outro através de uma conexão.
- **Telnet:** O Telnet fornece acesso por meio de uma sessão de terminal a um computador remoto.
- **SMTP:** O protocolo de transferência de correio simples é usado para enviar mensagens entre computadores.

Representação de Dados Externos e Empacotamento

As informações armazenadas nos programas em execução são representadas como estruturas de dados - por exemplo, por conjuntos de objetos interconectados - considerando que a informação em mensagens consiste em sequências de *bytes*.

Independentemente da forma de comunicação utilizada, as estruturas de dados devem ser achatadas (convertidas em uma sequência de *bytes*) antes da transmissão e reconstruídas na chegada.

Os itens de dados individuais transmitidos nas mensagens podem ser valores de dados de muitos tipos diferentes, e nem todos os computadores armazenam valores primitivos, como inteiros na mesma ordem.

A representação de números de ponto flutuante também difere entre arquiteturas. Existem duas variantes para a ordenação de números inteiros: a chamada ordem *big-endian*, em que o byte mais significativo vem primeiro; e ordem *little-endian*, em que vem por último.

Outra questão é o conjunto de códigos usados para representar caracteres: a maioria dos aplicativos em sistemas, como o UNIX por exemplo, usa caracteres na codificação ASCII, tomando um *byte* por caractere, enquanto o padrão Unicode permite a representação de textos em muitos idiomas diferentes e leva dois *bytes* por caractere.

Marshalling é o processo de coleta de dados e montagem em uma forma adequada para transmissão em uma mensagem. *Unmarshalling* é o processo de desmontá-los na chegada para produzir uma coleção equivalente de itens de dados no destino. Assim, o empacotamento consiste na tradução de itens de dados estruturados e valores primitivos em uma representação de dados externos.

Três abordagens alternativas para representação de dados externos e empacotamento são discutidas quando examinamos a abordagem do Google para representar dados estruturados:

- Representação de dados através da CORBA, que se preocupa com uma representação para os tipos estruturados e primitivos que podem ser passados como os argumentos e resultados de invocações de métodos remotos.
- Serialização de objetos do Java, que se preocupa com o nivelamento e a representação de dados de qualquer objeto único, ou árvore de objetos, que possam precisar ser transmitidos em uma mensagem ou armazenado em um disco. É para uso somente pelo Java.
- XML (*Extensible Markup Language*), que define um formato textual para representar dados estruturados. Foi originalmente destinado a documentos contendo dados estruturados autodescritivos textuais - por exemplo, documentos acessíveis na *Web* - mas agora também é usado para representar os dados enviados em mensagens trocadas por clientes e servidores em serviços da *Web*.

O Google usa uma abordagem chamada buffers de protocolo para capturar representações de ambos dados armazenados e transmitidos. Há também interesse considerável em JSON (*JavaScript Object Notation*) como uma abordagem para representação de dados. Buffers de protocolo e JSON representam um passo em direção abordagens mais leves à representação de dados.

Representação Comum de Dados (CDR) da CORBA

CORBA CDR é a representação de dados externa definida com o CORBA 2.0. O CDR pode representar todos os tipos de dados que podem ser usados como argumentos e retornar valores em invocações remotas.

Estes consistem em 15 tipos primitivos, *short* (16 bits), *long* (32 bits), *float* (32 bits), *double* (64 bits), *char*, booleano (*TRUE*, *FALSE*), *octal* (8 bits), juntamente com uma variedade de tipos compostos.

Cada argumento, ou resultado, em uma invocação remota é representado por uma sequência de *bytes* na mensagem de invocação ou resultado. Tipos primitivos:

CDR define uma representação para *big-endian* e *little-endian* de ordenamentos.

Embora seja simples, O CORBA CDR pode representar qualquer estrutura de dados que possa ser composta do primitivo e tipos construídos, mas sem usar ponteiros.

O compilador da interface CORBA gera o empacotamento e operações de desmarque para os argumentos e resultados de métodos remotos das definições dos tipos de seus parâmetros e resultados.

Serialização de Objetos Java

No Java RMI, ambos os objetos e valores de dados primitivos podem ser passados como argumentos e resultados de invocações de método. Um objeto é uma instância de uma classe Java.

A desserialização consiste em restaurar o estado de um objeto ou um conjunto de objetos de formulário serializado.

Objetos Java podem conter referências a outros objetos. Quando um objeto é serializado, todos os objetos que ele referencia são serializados juntos para garantir que quando o objeto é reconstruído, todas as suas referências possam ser cumpridas no destino.

Para serializar um objeto, suas informações de classe são gravadas, seguidas pelos tipos e nomes de suas variáveis de instância. Se as variáveis de instância pertencerem a novas classes, suas informações de classe também devem ser escritas, seguidas dos tipos e nomes de suas variáveis de instância.

Este procedimento recursivo continua até que a informação da classe e tipos e nomes das variáveis de instância de todas as classes necessárias sejam escritos.

Serialização e desserialização são geralmente realizadas automaticamente pelo *middleware*, sem qualquer participação do programador da aplicação.

O Uso da Reflexão

A linguagem Java suporta reflexão, que é a capacidade de perguntar sobre as propriedades de uma classe, como os nomes e tipos de suas variáveis de instância e métodos.

Também permite que classes sejam criadas a partir de seus nomes e um construtor com tipos de argumentos fornecidos para serem criados para uma determinada classe.

Extensible Markup Language (XML)

XML é uma linguagem de marcação definida pelo *World Wide Web Consortium* (W3C) para uso geral na *Web*. Em geral, o termo linguagem de marcação refere-se à codificação textual que representa tanto um texto e detalhes, quanto sua estrutura ou sua aparência.

Tanto o XML quanto o HTML foram derivados do SGML (Padronizado Linguagem de Marcação Generalizada) [ISO 8879], uma linguagem de marcação muito complexa.

O HTML foi desenvolvido para definir a aparência das páginas da *web*. XML foi concebido para escrever documentos estruturados para a *Web*.

Itens de dados XML são marcados com *strings* de “marcação”. As *tags* são usadas para descrever a estrutura lógica dos dados e associar pares de valores de atributos a dados lógicos estruturais. Ou seja, em XML, as *tags* estão relacionadas à estrutura do texto que elas contêm, em contraste com o HTML, no qual as *tags* especificam como um navegador pode exibir o texto.

XML é usado para permitir que clientes se comuniquem com serviços da *Web* e para definir as interfaces e outras propriedades dos serviços desta. No entanto, XML também é usado em muitas outras formas, incluindo em sistemas de arquivamento e recuperação. Ainda que um arquivo XML possa ser maior que um binário, tem a vantagem de poder ser lido em qualquer computador.

Outros exemplos de usos de XML incluem a especificação de interfaces de usuário e codificação de arquivos de configuração em sistemas operacionais. XML é extensível, no sentido de que os usuários podem definir suas próprias *tags*, em contraste com HTML, que usa um conjunto fixo de *tags*.

No entanto, se um documento XML for destinado a ser usado por mais de um aplicativo, os nomes das *tags* devem ser acordados entre eles.

Algumas representações externas de dados (como a CDR do CORBA) não precisam ser autodescritivas porque é assumido que o cliente e o servidor trocando uma mensagem têm conhecimento prévio da ordem e os tipos de informação que ela contém.

Contudo, o XML foi planejado para ser usado por vários aplicativos para finalidades diferentes, e o fornecimento de *tags*, juntamente com o uso de namespaces para definir o significado das *tags*, tornou isso possível.

Além disso, o uso de *tags* permite que os aplicativos selecionem as partes de um documento que ele precisa processar: ele não será afetado pela adição de informações relevantes para outras aplicações.

Documentos XML, sendo textuais, podem ser lidos por humanos. Na prática, a maioria dos XML documentos são gerados e lidos por *software* de processamento XML, mas a capacidade de ler XML pode ser útil quando as coisas dão errado. Além disso, o uso de texto torna o XML independente de qualquer plataforma específica. O uso de um texto, em vez de uma representação binária, juntamente com o uso de tags, torna as mensagens grandes, então elas requerem tempos de processamento e transmissão mais longos, além de mais espaço para armazenamento.

Como em HTML, a estrutura de um documento XML é definida por pares de tags entre colchetes angulares.

Um elemento em XML consiste em uma parte dos dados de caracteres cercados por *tags* de início e fim correspondentes. A capacidade de um elemento para incluir outro elemento permite que dados hierárquicos sejam representados.

Um arquivo XML contém atributos que pode incluir opcionalmente pares de nomes de atributos associados e valores como `id = "123456789"`.

Um esquema XML define os elementos e atributos que podem aparecer em um documento, como os elementos são aninhados e a ordem e número de elementos, e se um elemento está vazio ou pode incluir texto.

A intenção é que uma única definição de esquema possa ser compartilhada por muitos documentos. Um documento XML definido para estar em conformidade com um esquema específico pode também ser validado por meio desse esquema.

Definições de tipo de documento (DTDs) são fornecidas como parte da especificação XML 1.0 para definir a estrutura de documentos XML e ainda são amplamente utilizadas para esse fim.

Analisadores e geradores XML estão disponíveis para a maioria das linguagens de programação comumente usadas.

Referências de Objetos Remotos

Esta seção se aplica somente às linguagens como Java e CORBA que suportam modelo de objeto distribuído. Não é relevante para XML. Quando um cliente invoca um método em um objeto remoto, uma mensagem de chamada é enviada para o processo do servidor que hospeda o objeto remoto. Esta mensagem precisa especificar um objeto específico e ter seu método chamado. Uma referência de objeto remoto é um identificador para um objeto remoto que seja válido em todo o sistema distribuído.

Comunicação Multicast

A troca de mensagens entre pares não é o melhor modelo de comunicação de um processo a um grupo de outros processos, o que pode ser necessário, por exemplo, quando o serviço é implementado como um número de diferentes

processos em diferentes computadores, talvez para fornecer tolerância a falhas ou para aumentar a disponibilidade.

Uma operação *multicast* é mais apropriada - esta é uma operação que envia uma única mensagem de um processo para cada um dos membros de um grupo de processos, geralmente de forma que os membros do grupo sejam transparentes para o remetente. Existe uma gama de possibilidades no desejado comportamento de um *multicast*. O protocolo *multicast* mais simples não oferece garantias sobre entrega de mensagens ou pedidos.

As mensagens *multicast* fornecem uma infraestrutura útil para a construção de sistemas com as seguintes características:

1. Tolerância a falhas baseada em serviços replicados;
2. Descobrindo serviços em rede espontânea;
3. Melhor desempenho através de dados replicados;
4. Propagação de notificações de eventos.

Um exemplo da comunicação com o uso de *Multicast* pode ser feito no Facebook, quando alguém muda seu *status*, todos os seus amigos recebem notificações.

Confiabilidade e Ordenação de *Multicast*

O *multicast* também pode estar sujeito a falhas, um datagrama enviado de um roteador *multicast* para outro pode ser perdido, impedindo que todos os destinatários além desse roteador recebam a mensagem.

Além disso, um *multicast* em uma rede de área local usa os recursos de *multicast* da rede para permitir que um único datagrama chegue a vários destinatários.

Outro fator é que qualquer processo pode falhar. Se um roteador *multicast* falhar, o grupo membros além desse roteador não receberá a mensagem *multicast*, embora os membros podem fazê-lo.

Ordenar é outro problema. Pacotes IP enviados por uma inter-rede podem não necessariamente chegar na ordem em que foram enviados, com o possível efeito de que algum grupo membros receba datagramas de um único remetente em uma ordem diferente de outro grupo membros.

Alguns exemplos dos efeitos de confiabilidade e ordenação:

1. Tolerância a falhas baseada em serviços replicados;
2. Descoberta de serviços em rede espontânea;
3. Melhor desempenho através de dados replicados.

Estes exemplos sugerem que algumas aplicações requerem um protocolo *multicast* que é mais confiável do que IP *multicast*. Em particular, há uma necessidade

de *multicast* confiável, em que qualquer mensagem transmitida é recebida por todos os membros de um grupo ou por nenhum deles.

Virtualização de Rede

A força dos protocolos de comunicação da Internet é que eles fornecem um conjunto muito eficaz de blocos de construção para a construção de *software* distribuído. No entanto, uma gama crescente de diferentes classes de aplicação (incluindo, por exemplo, compartilhamento de arquivos *peer-to-peer* e Skype) coexiste na Internet.

Isto tornaria impraticável tentar alterar os protocolos da Internet para atender a cada um dos muitos aplicativos em execução sobre eles - o que pode melhorar um deles, pode ser prejudicial para outro. Além disso, o serviço de transporte IP é implementado em um grande e continuamente crescente número de tecnologias de rede. Esses dois fatores levaram ao interesse em virtualização de rede.

Virtualização de rede preocupa-se com a construção de muitas redes virtuais diferentes em uma rede existente, como a Internet. Cada rede virtual pode ser projetada para suportar um aplicativo distribuído em particular.

Redes de Sobreposição

Uma rede de sobreposição é uma rede virtual que consiste em nós e links virtuais, que fica em cima de uma rede subjacente (como uma rede IP) e oferece algo que não é.

Isso leva a uma ampla variedade de tipos de sobreposição e tem as seguintes vantagens:

- Novos serviços de rede;
- Múltiplas sobreposições.

As desvantagens são que as sobreposições introduzem um nível extra de indireção e aumentam a complexidade dos serviços de rede quando comparado, por exemplo, com a arquitetura relativamente simples de redes TCP/IP.

Um Exemplo de Rede de Sobreposição

O Skype é um aplicativo *peer-to-peer* que oferece Voz sobre IP (VoIP). Inclui também mensagens instantâneas, videoconferência e interfaces para o serviço de telefonia padrão através do SkypeIn e do SkypeOut.

O Skype é um excelente estudo de caso sobre o uso de redes de sobreposição no mundo real (e em grande escala), indicando como a funcionalidade avançada pode ser fornecida em uma forma específica da aplicação e sem modificação da arquitetura central do Internet.

O Skype é baseado em uma infraestrutura *peer-to-peer* que consiste em máquinas comuns dos usuários (referidas como *hosts*) e super-nós. Os super-nós são *hosts* comuns do Skype que possuem recursos suficientes para executar suas ações.

Resumo

A primeira seção deste item mostrou que os protocolos de transmissão da Internet são dois blocos de construção alternativos, a partir dos quais os protocolos de aplicativo podem ser construídos.

Existe um *trade off* interessante entre os dois protocolos: o UDP fornece uma facilidade de passagem de mensagens que sofre de falhas de omissão, mas não tem built in penalidades de desempenho, por outro lado, em boas condições, o TCP garante a entrega, mas à custa de mensagens adicionais e maior latência e custos de armazenamento.

A segunda seção apresentou três estilos alternativos de empacotamento: CORBA, Java e XML.

Mensagens *multicast* são usadas na comunicação entre os membros de um grupo de processos. A *multicast IP* fornece um serviço de *multicast* para redes locais e a Internet. Esta forma de *multicast* tem a mesma semântica de falha que os datagramas UDP, mas apesar de sofrer de falhas de omissão, é uma ferramenta útil para muitas aplicações de *multicast*. Algumas outras aplicações têm requisitos mais fortes - em particular, a entrega *multicast* deve ser atômica, isto é, deve entregar tudo ou nada.

Outros requisitos sobre *multicast* estão relacionados com a ordenação de mensagens, o mais forte dos quais requer que todos os membros de um grupo recebam todas as mensagens na mesma ordem.

O *multicast* também pode ser suportado por redes de sobreposição nos casos em que, por exemplo, O *multicast IP* não é suportado. Geralmente, contudo, as redes de sobreposição oferecem um serviço de virtualização da arquitetura de rede, permitindo serviços de rede especializados criados em cima da infraestrutura de rede subjacente (por exemplo, UDP ou TCP).

As redes de sobreposição abordam parcialmente os problemas associados às questões fim-a-fim, permitindo a geração de mais abstrações de rede específicas de aplicativos.

RPC - Chamada de Procedimento Remoto

A abordagem de chamada de procedimento remoto (RPC) estende a programação comum abstração da chamada de procedimento para ambientes distribuídos, permitindo uma chamada processo para chamar um procedimento em um nó remoto como se fosse local.

Chamada de método remoto (RMI) é semelhante ao RPC, mas para objetos distribuídos, com benefícios adicionais em termos de uso de conceitos de programação orientada a objetos em sistemas distribuídos e também estendendo o conceito de um objeto de referência para os ambientes distribuídos globais, permitindo o uso de referências a objetos como parâmetros em invocações remotas.

Introdução

Este item trata de como os processos se comunicam em um sistema distribuído, examinando, em particular, os paradigmas de invocação remota:

- Os protocolos de solicitação e resposta representam um padrão no topo da passagem de mensagens e suporte a troca bidirecional de mensagens encontrada na computação cliente-servidor.
- O exemplo mais antigo e talvez o mais conhecido de um programa mais amigável do modelo foi a extensão do modelo de chamada de procedimento convencional para sistemas (RPC), que permite ao cliente de programas chamar procedimentos de forma transparente em programas de servidor em execução em separados processos e geralmente em diferentes computadores do cliente.

Protocolos de Solicitação e Resposta

Esta forma de comunicação é projetada para apoiar os papéis e trocas de mensagens em interações cliente-servidor típicas. No caso normal, a comunicação de solicitação-resposta é síncrona porque o processo do cliente bloqueia até que a resposta chegue do servidor.

O HTTP é um exemplo de um protocolo de solicitação e resposta. É usado pelos clientes do navegador da Web para fazer solicitações a servidores da Web e receber respostas deles.

HTTP é um protocolo que especifica as mensagens envolvidas em uma solicitação-resposta troca, os métodos, argumentos e resultados, e as regras para representa-los nas mensagens. Suporta um conjunto fixo de métodos (GET, PUT, POST, etc.) que são aplicáveis a todos os recursos do servidor.

Solicitações e respostas são empacotadas em mensagens como *strings* de texto ASCII, mas recursos podem ser representados como sequências de *bytes* e podem ser compactados.

O *Multipurpose Internet Mail Extensions* (MIME), especificado no RFC 2045, é um padrão para o envio de dados multiparte contendo, por exemplo, texto, imagens e som em mensagens de e-mail.

Chamada de Procedimento Remoto

O conceito de uma chamada de procedimento remoto (RPC) representa grande avanço intelectual em computação distribuída, com o objetivo de tornar a programação de sistemas distribuídos similares, se não idênticos, à programação - isto é, alcançar um alto nível de transparência na distribuição.

Problemas de *Design* para RPC

Antes de analisar a implementação dos sistemas RPC, analisamos três pontos que são importantes na compreensão deste conceito:

Programação com interfaces

A maioria das linguagens de programação modernas fornece um meio de organizar um programa como um conjunto de módulos que podem se comunicar uns com os outros. A comunicação entre os módulos pode ser feita por meio de chamadas de procedimento entre os módulos, ou pelo acesso direto às variáveis em outro módulo. Para controlar as possíveis interações entre módulos, uma interface explícita é definida para cada módulo. A interface de um módulo especifica os procedimentos e as variáveis que podem ser acessadas de outros módulos. Módulos são implementados de modo a esconder todas as informações sobre eles, exceto o que está disponível através de sua interface.

Linguagens de definição de interface

Um mecanismo RPC pode ser integrado a uma determinada linguagem de programação se incluir uma notação adequada para definir as interfaces, permitindo que os parâmetros de entrada e saída sejam mapeados no uso normal do idioma parâmetros. Essa abordagem é útil quando todas as partes de um aplicativo distribuído podem ser escritas na mesma língua.

Semântica de chamadas RPC

Os protocolos de solicitação-resposta podem ser implementados de maneiras diferentes para fornecer diferentes garantias de entrega.

As chamadas de procedimento remoto são mais vulneráveis a falhas do que as locais, já que envolvem uma rede, outro computador e outro processo. Seja qual for a semântica escolhida, há sempre a chance de que nenhum resultado seja recebido, e em caso de falha, é impossível distinguir entre falha da rede e do processo do servidor remoto. Isso exige que os clientes que fazem chamadas remotas possam recuperar de tais situações.

Implementação do RPC

Os componentes de software necessários para implementar o RPC são os seguintes:

- Um serviço inclui um procedimento de acesso para cada procedimento na interface de serviço.
- Um módulo de comunicação para o servidor.

O RPC é geralmente implementado em um protocolo de solicitação-resposta. O conteúdo das mensagens de solicitação e resposta é o mesmo para os protocolos de solicitação e resposta.

Chamada de Método Remoto

A invocação remota de métodos (RMI, *Remote Method Invocation*) está intimamente relacionada ao RPC, mas estendida para o mundo de objetos distribuídos. No RMI, um objeto de chamada pode invocar um método em um objeto remoto. Assim como no RPC, os detalhes subjacentes geralmente são ocultados do usuário.

As semelhanças entre o RMI e o RPC são as seguintes:

- Ambos suportam programação com interfaces, com os benefícios resultantes que decorrem desta abordagem.
- Ambos são tipicamente construídos sobre os protocolos de solicitação-resposta e podem oferecer um intervalo de semântica de chamadas, como pelo menos uma vez e no máximo uma vez.
- Ambos oferecem um nível semelhante de transparência - ou seja, chamadas locais e remotas empregam a mesma sintaxe, mas as interfaces remotas normalmente expõem a distribuição natureza da chamada subjacente, por exemplo, suportando exceções remotas.

As seguintes diferenças levam a uma maior expressividade quando se trata da programação de aplicações e serviços distribuídos complexos.

- O programador é capaz de usar todo o poder expressivo de programação orientada a objetos no desenvolvimento de software de sistemas distribuídos, incluindo uso de objetos, classes e herança, e também pode empregar metodologias de design orientadas a objetos e ferramentas associadas.
- Com base no conceito de identidade de objetos em sistemas orientados a objetos, todos os objetos em um sistema baseado em RMI têm referências de objeto exclusivas (sejam elas locais ou remotas), tais referências de objeto também podem ser passadas como parâmetros e semântica de passagem de parâmetros significativamente mais rica que no RPC.

Problemas de *Design* para RMI

Como mencionado acima, o RMI compartilha os mesmos problemas de *design* que o RPC em termos de programação com interfaces, semântica de chamadas e nível de transparência.

As principais questões estão associadas a seguir:

- O modelo de objeto;
- Referências de objetos;
- Interfaces;
- Ações;
- Exceções;
- *Garbage Collection*.

Objetos Distribuídos

O estado de um objeto consiste nos valores de sua instância variáveis. No paradigma baseado em objetos, o estado de um programa é particionado em partes, cada uma delas associada a um objeto. Sistemas de objetos distribuídos podem adotar a arquitetura cliente-servidor. Nesse caso, objetos são gerenciados por servidores e seus clientes invocam seus métodos usando invocação de método.

Objetos distribuídos podem assumir outros modelos de arquitetura. Por exemplo, objetos podem ser replicados para obter os benefícios usuais de tolerância a falhas e desempenho, e os objetos podem ser migrados com vista a melhorar o seu desempenho e disponibilidade.

O Modelo de Objeto Distribuído

Cada processo contém uma coleção de objetos, alguns dos quais podem receber invocações locais e remotas, enquanto o outro objeto pode receber apenas invocações locais. Invocação de método entre objetos em diferentes processos, seja no mesmo computador ou não, são conhecidas como modelo de objeto distribuído.

Resumo

Este item discutiu três paradigmas para programação distribuída - solicitação-resposta protocolos, chamadas de procedimento remoto e invocação remota de método. Todos esses paradigmas fornecem mecanismos para entidades independentes distribuídas (processos, objetos componentes ou serviços) para se comunicarem diretamente uns com os outros.

Os protocolos de solicitação e resposta fornecem suporte leve e mínimo para o cliente-servidor Informática. Esses protocolos costumam ser usados em ambientes onde as despesas indiretas à comunicação devem ser minimizadas - por exemplo, em sistemas embarcados. Seu mais comum papel é apoiar RPC ou RMI, como discutido abaixo. A abordagem de chamada de procedimento remoto foi um avanço significativo em sistemas, fornecendo suporte de alto nível para programadores, estendendo o conceito de chamada de procedimento para operar em um ambiente de rede. Isso fornece níveis importantes de transparência em sistemas distribuídos. No entanto, devido à sua falha diferente e características de desempenho e à possibilidade de acesso simultâneo a servidores, não é necessariamente uma boa ideia fazer com que chamadas de procedimentos remotos pareçam exatamente as mesmas que chamadas locais. Chamadas de procedimento remoto fornecem um intervalo de semânticas de invocação, de talvez, invocações através da semântica, no máximo uma vez.

O modelo de objeto distribuído é uma extensão do modelo de objeto local usando linguagens de programação baseadas em objetos. Objetos encapsulados formam componentes úteis em um sistema distribuído, já que o encapsulamento os torna inteiramente responsáveis pelo gerenciamento de seu próprio estado e a invocação local de métodos pode ser estendida para invocação remota.

Cada objeto em um sistema distribuído possui uma referência de objeto remoto identificador e uma interface remota que especifica quais de suas operações podem ser invocadas remotamente.

As implementações de *middleware* do RMI fornecem componentes (incluindo proxies, esqueletos e despachantes) que escondem os detalhes de empacotamento, passagem de mensagens e localizam objetos remotos de programadores de clientes e servidores. Esses componentes podem ser gerados por um compilador de interface. Java RMI estende a chamada local para invocação remota usando a mesma sintaxe, mas as interfaces remotas devem ser especificadas pela extensão uma interface chamada Remoto, fazendo com que cada método lance uma *RemoteException*. Este garante que os programadores saibam quando fazem invocações remotas ou implementam objetos remotos, permitindo-lhes manipular erros ou projetar objetos adequados acesso simultâneo.

Comunicação Indireta

Este item completa o nosso roteiro pelos paradigmas de comunicação, baseia-se em comunicação entre processos. A essência da comunicação indireta é comunicar através de um intermediário e, portanto, não têm nenhum acoplamento direto entre o remetente e um ou mais receptores.

Introdução

Este item conclui a avaliação dos paradigmas de comunicação examinando comunicação indireta, com base nos estudos de comunicação entre processos e invocação remota, respectivamente.

Todos os problemas da ciência da computação podem ser resolvidos por outro nível de indireção. Em termos de sistemas distribuídos, o conceito de indireção é cada vez mais aplicado a paradigmas de comunicação.

A comunicação indireta é definida como comunicação entre entidades em um sistema distribuído através de um intermediário sem acoplamento direto entre o remetente e o(s) receptor (es). A natureza precisa do intermediário varia de abordagem para abordagem, além disso, é necessário o acoplamento e pode variar significativamente entre os sistemas. Observe o plural opcional associado ao receptor; isso significa que muitos paradigmas de comunicação indireta apoiam explicitamente a comunicação um-para-muitos.

As técnicas consideradas são todas baseadas em um acoplamento direto entre um remetente e um receptor, e isso leva a certa rigidez no sistema em termos de lidar com a mudança.

Além disso, sistemas desenvolvidos usando comunicação indireta podem ser mais difíceis de gerenciar.

Comunicação em Grupo

A comunicação em grupo fornece nosso primeiro exemplo de paradigma de comunicação indireta. A comunicação em grupo oferece um serviço pelo qual uma mensagem é enviada para um grupo e então esta mensagem é entregue a todos os membros do grupo.

Com as garantias adicionadas, a comunicação em grupo é para *multicast IP*, o TCP é utilizado para o serviço ponto-a-ponto em IP.

A comunicação em grupo é um importante bloco de construção para sistemas distribuídos e sistemas distribuídos particularmente confiáveis.

O Modelo de Programação

Na comunicação em grupo, o conceito central é o de um grupo com grupo associado associação, em que os processos podem ingressar ou sair do grupo. Os processos podem enviar uma mensagem para este grupo e tê-lo propagado para todos os membros do grupo com garantias em termos de confiabilidade e pedidos. Assim, os implementos de comunicação em grupo comunicação multicast, em que

uma mensagem é enviada a todos os membros do grupo por uma única operação. Comunicação para todos os processos do sistema, em oposição a um subgrupo deles, é conhecida como *broadcast*, enquanto a comunicação para um único processo é conhecida como *unicast*.

A característica essencial da comunicação em grupo é que um processo emite apenas uma operação *multicast* para enviar uma mensagem para cada um dos grupos de processos, em vez de emitir várias operações de envio para processos individuais.

Problemas de Implementação

As questões de implementação para serviços de comunicação em grupo, discutindo a propriedades do serviço multicast subjacente em termos de confiabilidade e ordenação, e também o papel fundamental do gerenciamento de associação de grupo em ambientes dinâmicos, onde processos podem entrar e sair ou falhar a qualquer momento.

Na comunicação em grupo, todos os membros de um grupo devem receber cópias das mensagens enviadas ao grupo, geralmente com entrega garantida. As garantias incluem acordo sobre o conjunto de mensagens que todo processo no grupo deve receber e na ordem de entrega entre os membros do grupo.

Os sistemas de comunicação em grupo são extremamente sofisticados. Mesmo *multicast IP*, que fornece garantias mínimas de entrega, requer um grande esforço de engenharia.

A confiabilidade na comunicação um-para-um pode ser definida em termos de duas propriedades: integridade e validade. A interpretação para multicast confiável baseia-se nessas propriedades, com integridade definida da mesma forma em termos de entregar a mensagem corretamente no máximo uma vez, e validade, garantindo que uma mensagem enviada será eventualmente entregue.

Além das garantias de confiabilidade, a comunicação em grupo exige garantias extras em termos da ordem relativa de mensagens entregues para vários destinos.

Sistemas de Publicação-Assinatura

Sistemas de publicação-assinatura, às vezes também chamados de sistemas distribuídos baseados em eventos.

Um sistema de publicação-assinatura é um sistema em que os editores publicam eventos para um serviço de eventos e os assinantes manifestam interesse em eventos específicos através de assinaturas que podem ser padrões arbitrários sobre os eventos estruturados. Por exemplo, um assinante pode expressar interesse em todos os eventos relacionados a um determinado livro, como disponibilidade de uma nova edição ou atualizações para o site relacionado.

Sistemas de publicação-assinatura são usados em uma grande variedade de domínios de aplicação, particularmente aqueles relacionados à grande escala de divulgação de eventos.

Exemplos incluem:

- Sistemas de informação financeira;
- Outras áreas com feeds ao vivo de dados em tempo real (incluindo feeds RSS);
- Apoio ao trabalho cooperativo, onde vários participantes precisam ser informados de eventos de interesse comum;
- Suporte para computação onipresente, incluindo o gerenciamento de eventos da infraestrutura onipresente (por exemplo, eventos de localização);
- Um amplo conjunto de aplicativos de monitoramento, incluindo monitoramento de rede no Internet.

O Modelo de Programação

O modelo de programação em sistemas de publicação-assinatura é baseado em um pequeno conjunto de operações.

Alguns sistemas complementam o conjunto de opções, introduzindo o conceito de anúncio. Com os anúncios, os editores têm a opção de declarar a natureza dos eventos futuros por meio de uma operação de propaganda.

Problemas de Implementação

A tarefa de um sistema de publicação-assinatura é clara: garantir que os eventos sejam entregues eficientemente a todos os assinantes que possuam filtros definidos evento. Adicionado a isso, pode haver requisitos adicionais em termos de segurança, escalabilidade, manuseio de falhas, simultaneidade e qualidade de serviço. Isso faz com que a implementação de sistemas de publicação-assinatura seja bastante complexa, e esta tem sido uma área de intensa investigação na comunidade de pesquisa.

Filas de Mensagens

Filas de mensagens são uma importante categoria de sistemas de comunicação indireta. Considerando que grupos e publica-subscriver forneça um estilo de comunicação um-para-muitos, filas de mensagens fornecem serviço ponto-a-ponto usando o conceito de uma fila de mensagens como uma indireção, alcance as propriedades desejadas de desacoplamento do espaço e do tempo. Eles são ponto-a-ponto em que o remetente coloca a mensagem em uma fila e, em seguida, é removido por um único processo. As filas de mensagens também são referidas como *Middleware Orientado por Mensagens*. Isto é uma grande classe de *middleware* comercial com implementações importantes, incluindo WebSphere MQ, MSMQ da Microsoft e Oracle Advanced Enfileiramento (AQ) do Oracle.

O Modelo de Programação

O modelo de programação oferecido pelas filas de mensagens é muito simples. Ele oferece uma abordagem à comunicação em sistemas distribuídos através de filas. Em particular, os processos do produtor podem enviar mensagens para uma fila específica e outros (consumidor) processos podem receber mensagens dessa fila.

Uma propriedade crucial dos sistemas de fila de mensagens é que as mensagens são persistentes - isto é, as filas de mensagens armazenarão as mensagens indefinidamente (até serem consumidas) e também enviará as mensagens para o disco para permitir uma entrega confiável.

Problemas de Implementação

O principal problema de implementação dos sistemas de enfileiramento de mensagens é a escolha entre implementações centralizadas e distribuídas do conceito. Algumas implementações são centralizadas, com uma ou mais filas de mensagens gerenciadas por um gerenciador de filas em determinado nó. A vantagem deste esquema é a simplicidade, mas tais gerentes podem se tornar componentes bastante pesados, tendo o potencial para se tornar um gargalo ou um ponto único de falha. Como resultado, implementações mais distribuídas foram propostas. Para ilustrar as arquiteturas distribuídas, consideramos brevemente a abordagem adotada no WebSphere MQ como representante do estado da arte nessa área.

O WebSphere MQ é um *middleware* desenvolvido pela IBM sobre o conceito de filas de mensagens, oferecendo uma indireção entre os remetentes e os destinatários de mensagens. Filas no WebSphere MQ são gerenciadas por gerenciadores de filas que hospedam e gerenciam filas e permitem que aplicativos acessem filas através da Message Queue Interface (MQI). O MQI é uma interface relativamente simples permitindo que os aplicativos realizem operações como conectar ou desconectar a partir de uma fila (MQCONN e MQDISC) ou enviando / recebendo mensagens de / para uma fila (MQPUT e MQGET). Vários gerenciadores de filas podem residir em um único servidor físico. Aplicativos clientes que acessam um gerenciador de filas podem residir no mesmo servidor físico. Geralmente, porém, eles estarão em máquinas diferentes e devem se comunicar com o gerenciador de filas por meio do que é conhecido como canal do cliente. Os canais de clientes adotam o conceito bastante familiar de um proxy, em que os comandos MQI são emitidos no proxy e, em seguida, enviados de forma transparente para o gerenciador de filas para execução usando RPC.

Abordagens de Memória Compartilhada

Alguns paradigmas de comunicação indireta oferecem uma abstração de memória compartilhada. Considerando que a memória compartilhada distribuída opera no nível da leitura escrevendo *bytes*, oferecem uma perspectiva de nível superior na forma de dados estruturados.

Memória Compartilhada Distribuída

A memória compartilhada distribuída (DSM) é uma abstração usada para compartilhar dados entre computadores que não compartilham memória física. Processos acessam o DSM por meio de leituras e atualizações para o que parece ser uma memória comum em seu espaço de endereço. No entanto, um sistema de tempo de execução subjacente garante de forma transparente que os processos de execução em diferentes computadores observem as atualizações feitas umas pelas outras. É como se os processos acessassem uma única memória compartilhada, mas, na verdade, a memória física é distribuída.

O ponto principal do DSM é que ele poupa ao programador as preocupações da mensagem passando ao escrever aplicativos que poderiam precisar usá-lo. O DSM é principalmente uma ferramenta para aplicativos paralelos, ou para qualquer aplicativo distribuído ou grupo de aplicativos em que itens de dados compartilhados individuais podem ser acessados diretamente. O DSM é, em geral, menos apropriado em sistemas cliente-servidor, em que os clientes normalmente visualizam recursos mantidos pelo servidor como dados abstratos e acessá-los por solicitação (por razões de modularidade e proteção).

A passagem de mensagens não pode ser totalmente evitada em um sistema distribuído: na ausência de memória física compartilhada, o suporte ao tempo de execução do DSM deve enviar atualizações mensagens entre computadores.

A importância do DSM cresceu primeiro ao lado do desenvolvimento da memória compartilhada multiprocessadores. Muita pesquisa entrou em investigando algoritmos adequados para computação paralela nesses multiprocessadores.

Em multiprocessadores de memória distribuída e clusters de computação, os processadores não compartilham memória, conectados por uma rede de alta velocidade.

Estes sistemas, como o uso geral de sistemas distribuídos, pode escalar para um número muito maior de processadores do que 64 ou mais do multiprocessador de memória.

Resumo

Este item examinou detalhadamente a comunicação indireta, complementando o estudo de paradigmas de invocação remota no item anterior. Nós definimos a comunicação indireta em termos de comunicação através de um intermediário, resultando desacoplamento entre produtores e consumidores de mensagens. Isso leva a interessantes propriedades, particularmente em termos de lidar com a mudança e estabelecer tolerantes a falhas estratégias.

Consideramos os estilos de comunicação indireta neste item:

- Comunicação em grupo;
- Sistemas de publicação/assinatura;
- Filas de mensagens;
- Memória compartilhada distribuída;

A discussão enfatizou suas semelhanças em termos de todos os recursos indiretos de comunicação através de formas de intermediários, incluindo grupos, canais ou tópicos, filas, memória compartilhada. Sistemas de publicação-assinatura baseados em conteúdo comunicar-se através do sistema de publicação-assinatura como um todo, com assinaturas efetivamente definindo canais lógicos gerenciados pelo roteamento baseado em conteúdo.

A maioria dos sistemas acima também oferece estilos de comunicação de um para muitos, sendo *multicast* em termos de serviços baseados em comunicação e acesso global dos valores compartilhados nas abstrações baseadas em estado. As exceções são enfileiramento de mensagens, que é fundamentalmente ponto-a-ponto (e, portanto, muitas vezes oferecido em combinação com sistemas de subscrição em *middleware* comercial), espaços de dupla, que podem ser um-para-muitos ou ponto a ponto, dependendo se os processos de recebimento usam a leitura ou operações, respectivamente.

Existem também diferenças de intenção nos vários sistemas. Comunicação em grupo é principalmente projetado para suportar sistemas distribuídos confiáveis e, portanto, a ênfase é fornecendo suporte algorítmico para confiabilidade e ordenação de entrega de mensagens.

Curiosamente, os algoritmos para garantir confiabilidade e ordenação (especialmente o último) podem ter um efeito negativo significativo sobre a escalabilidade por razões semelhantes para manter visualizações consistentes do estado compartilhado. Os sistemas de publicação/assinatura foram amplamente direcionados na divulgação de informações (por exemplo, em sistemas financeiros) e para empresas de Integração de aplicativos. Finalmente, as abordagens de memória compartilhada geralmente aplicadas em processamento paralelo e distribuído, incluindo na comunidade *Grid*.

Não foram consideradas questões relacionadas à qualidade de serviço, muitos sistemas de fila de mensagens oferecem suporte intrínseco para a confiabilidade na forma de transações.

Geralmente, no entanto, a qualidade do serviço continua sendo um desafio aos paradigmas de comunicação indireta. De fato, raciocinar sobre as propriedades de ponta a ponta do sistema dificultam essa questão, questões como comportamento do tempo ou segurança, e, portanto, esta é uma área importante para futuras pesquisas.

Material Complementar

Indicações para saber mais sobre os assuntos abordados nesta Unidade:

Vídeos

Sistemas Distribuídos - Aula 05 - Processos distribuídos

https://youtu.be/BkjXhh0_daY

Leitura

Comunicação entre Processos

<https://goo.gl/a2LPzi>

Invocação Remota

<https://goo.gl/Stb2J5>

Comunicação entre Processos

<https://goo.gl/opL8WQ>

Referências

GAGLIARDI, Gary. **Cliente/servidor**. São Paulo: Makron Books do Brasil, 1996.

STALLINGS, William. **Arquitetura e organização de computadores**: projeto para o desempenho. 8. ed. São Paulo: Pearson Prentice Hall, 2009. ISBN 9788576055648.

TANENBAUM, Andrew S.; Steen, Maarten van. **Sistemas Distribuídos**: princípios e paradigmas - 2ª edição. Pearson 416 ISBN 9788576051428.



Cruzeiro do Sul
Educatonal